

1. Lista – Gerenciador de Tarefas

Enunciado:

Imagine que você está desenvolvendo uma aplicação para uma pequena equipe de profissionais autônomos que trabalham de forma colaborativa em tarefas do dia a dia, como manutenção residencial, design gráfico, ou suporte técnico. Cada membro da equipe precisa manter sua lista pessoal de tarefas organizadas para não perder prazos e manter o foco,

A equipe está testando um assistente virtual simples que ajude os membros a registrar, visualizar e remover tarefas do seu dia de trabalho. Esse sistema será operado por comandos simples e será responsável por simular a ação de um pseudo agente (como um robô de tarefas pessoais) que entende e executa as ordens dadas.

Crie um programa que permita ao usuário adicionar, remover e listar tarefas. As tarefas devem ser armazenadas em uma **lista**. O usuário deve poder:

- Adicionar uma nova tarefa digitando o nome.
- Remover uma tarefa pelo número da posição.
- Exibir todas as tarefas na ordem em que foram adicionadas.

Objetivo didático:

Trabalhar com inserção, remoção e iteração em listas.

2. Fila – Atendimento em Clínica

Enunciado:

Uma clínica médica de pequeno porte está em processo de modernização dos seus sistemas. O objetivo é substituir o atendimento manual na recepção por um sistema automatizado com agentes virtuais simples que auxiliem na organização da fila de pacientes.

A clínica deseja um sistema que simule o comportamento de um agente recepcionista, que recebe os pacientes, os insere em uma fila de espera, chama o próximo da fila e informa o status atual da fila a qualquer momento.

Esse agente virtual não toma decisões médicas, mas apenas organiza a ordem de atendimento respeitando a regra **FIFO** (First In, First Out), ou seja, quem chega primeiro é atendido primeiro.

Simule uma **fila de atendimento** em uma clínica médica. O programa deve permitir:

- Adicionar o nome de um paciente à fila.
- Chamar o próximo paciente (remover da fila e mostrar quem foi chamado).
- Mostrar a fila atual.

Objetivo didático:

Compreender a estrutura **FIFO (First In, First Out)** e manipulação básica de filas.



3. Pilha – Navegador Web (Histórico de Navegação)



Enunciado:

Imagine que você está desenvolvendo parte da lógica de navegação de um navegador web inteligente, como um navegador minimalista controlado por comandos de texto. A navegação é operada por um pseudo agente navegador, responsável por manter o histórico das páginas acessadas e gerenciar o comando de “voltar” para a página anterior.

Esse pseudo agente precisa controlar o fluxo de páginas da seguinte forma:

- Quando o usuário acessa uma nova página, o agente a armazena no topo de uma pilha.
- Quando o usuário solicita “voltar”, o agente remove a página atual da pilha e retorna para a anterior.
- A qualquer momento, o agente pode exibir qual página está atualmente sendo visualizada.

Implemente uma simulação do botão “voltar” de um navegador. Para isso, use uma **pilha** onde cada nova página acessada é empilhada. O programa deve permitir:

- Acessar uma nova página (empilhar).
- Voltar para a página anterior (desempilhar).
- Mostrar a página atual no topo da pilha.

Objetivo didático:

Explorar a lógica de **LIFO (Last In, First Out)** com empilhamento e desempilhamento.